Advanced Adder Designs for High-Performance Digital Circuits

Ekansh Bajpai
Electrical Systems Engineering
University of Paderborn
ekansh@mail.upb.de

***Abstract*—In the current scenario of AI and ML, there is an increasing requirement for high-performance digital circuits. The increased performance is generally related to more speed, area optimization, and low power consumption. This report gives an insight into adder circuits in digital electronics, especially optimizing addition delay through hierarchical and parallel computation techniques. It starts with basic concepts, including binary addition principles, half adders, and full adders and their circuit implementations. Then, the report explores the application of generate-propagate logic and Manchester carry chains to minimize delay and increase power efficiency. Advanced design architectures like carry-save adders, carry-bypass adders, and lookahead adders are discussed, along with their performance trade-offs and critical path delays. To conclude, it provides a detailed summary of the advantages and disadvantages of adder circuit designs and the scenarios where one is preferred over another.**

***Index Terms*—binary addition, ripple carry, carry-save, generate-propagate, lookahead, carry-bypass**

I. Introduction

One of the basic arithmetic operations in digital systems, such as processors and microcontrollers, is binary addition. Efficient design and implementation of these adders improve the performance in terms of more speed, minimized circuit area, and lower power consumption. As digital applications grow in complexity, from low-power IoT devices to high-performance AI accelerators, optimizing adder architectures has become increasingly vital. This report is divided into three main sections. The first part discusses the fundamental building blocks of the binary addition process, which are half adders (HA) and full adders (FA), along with CMOS transistors. These foundational circuits form the basis of more complex adder architectures. In the second part, the report examines the more complex adder designs like generate-propagate (G-P logic) and lookahead addition. Each of these designs addresses specific challenges related to critical path delay, word length scalability, and power efficiency. It also discusses their fundamental structure and limitations. For instance, the ripple carry adder, provides simple implementation but has a drawback from linear delay dependence on word length, while other complex architectures such as lookahead adders focus to reduce this delay through hierarchical and parallel computation techniques. These adder designs serve as the central aspect of the report. In the end, the report summarizes the various designs along with the discussion on their advantages and disadvantages.

II. Basic Principles

Before discussing more complex binary adder designs, it is important to understand the basic terminologies and principles of digital circuit arithmetic. Understanding these building blocks is essential as they contribute to adder performance by providing insight for more optimized design workflows.

A. Binary Addition

Binary addition is the foundation of many operations in digital circuits, especially an ALU (Arithmetic Logic Unit) for arithmetic operations. Binary addition in digital systems is done using the same approach as decimal long addition [1], but in this case, the nature of operands is binary. The sum-carry logic is much more straightforward in binary addition. Basic binary operations used in digital arithmetic are addition and multiplication [2].

TABLE I
Binary Addition Table [1]

Position		4	3	2	1	0
Carries		1	0	1	1	
Operand 1		1	1	0	0	1
Operand 2		0	1	0	1	1
Sum	1	0	0	1	0	0

Table I shows an example of the binary addition of two decimal numbers, 25 and 11. The binary equivalents for these numbers are 11001 and 01011, respectively. Both these decimal numbers are 5-bit binary numbers. The result of the decimal addition is 36, and the decimal equivalent is 100100. Generally, the sum of n-bit operands gives a result of (n+1)-bits, but for some cases, the result can also be n-bit. As evident from Table I, when adding LSB (Least Significant Bit), there are only two operands and no carry bit, resulting in one sum bit and one carry bit. While performing addition operations on a bit position other than LSB, three bits are added, i.e., two operand bits of the current bit position and one carry bit, known as carry-in (C_{in}), from the previous bit position. In this case, the result is also a sum bit and a carry bit [1].

B. CMOS

CMOS (Complementary Metal Oxide Semiconductor) forms the basis of most current logic gates that help to design

digital circuits for adders. It helps construct efficient, reliable, and scalable logic gates. Each transistor has a sandwich-like structure created by superimposing multiple layers of insulating and conducting materials. Each transistor comprises of an insulating silicon dioxide layer, a gate, and a silicon substrate. The gate controls the flow of current by creating an electric field, from the source to the drain. Since transistor operation is controlled by electric fields, they are also known as MOSFETs (Metal Oxide Semiconductor Field Effect Transistors) or FETs. This technology provides enhanced performance by operating at a much lower voltage, which provides more energy efficiency. CMOS technology uses two types of transistors, pMOS and nMOS, to design circuits [2].

1) pMOS Transistor: In case of pMOS transistor, there is a positive voltage at the body. pMOS turns ON when voltage at gate is low and turns OFF when the voltage at the gate is high [2]. A cross section of a pMOS transistor given in the Figure 1.

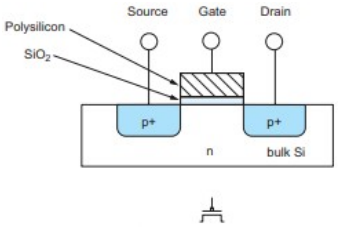


Fig. 1. pMOS Transistor [2]

2) nMOS Transistor: In case of nMOS transistor, the body is connected to ground. nMOS is ON when the gate voltage is high and OFF when the voltage at gate is low [2]. A cross-section of a nMOS is given in Figure 2.

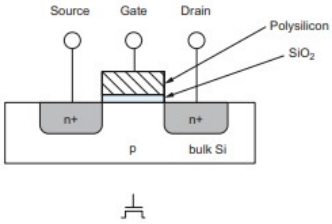


Fig. 2. nMOS Transistor [2]

C. Half Adder

The half adder (HA) is the simplest digital circuit design used for adding two binary numbers. It is the case where there is no C_{in} bit involved. Only two operand bits at a bit position are added, generally the LSB. The circuit results in two output bits: the sum bit (S) that becomes part of the sum result, and the carry-out C_{out} bit that goes as C_{in} to the next bit position [1].

Table II shows the truth table for a half adder with A and B as input bits and S and C_{out} as the outputs. The boolean equations for the S and C_{out} are as follows [1]:

$$S = A \oplus B \tag{1}$$

TABLE II
TRUTH TABLE FOR HALF ADDER [1]

A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$C = A \cdot B \tag{2}$$

The gate circuit for the C_{out} uses four transistors for implementation, assuming complements of input are available, as shown in Figure 3. The gate circuit for S has two input XOR gates and contains the complemented inputs [1]. Figure 4 shows the implementations using the least number of transistors.

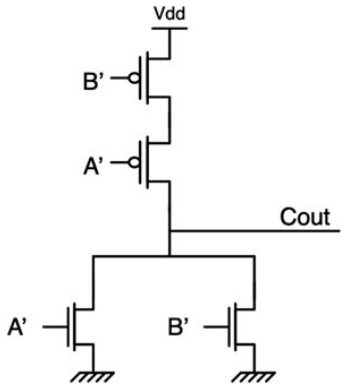


Fig. 3. C_{out} gate implementation for HA [1]

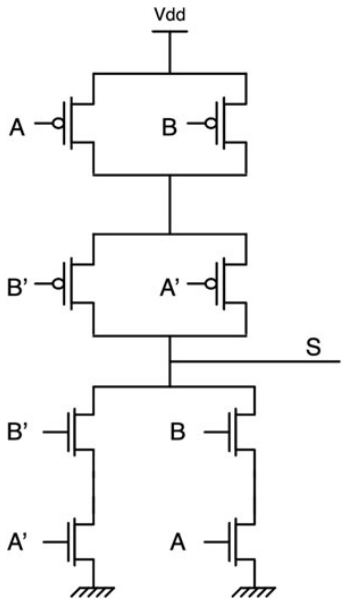


Fig. 4. S gate implementation for HA [1]

One of the drawbacks of a half-adder circuit is that it does not account for C_{in} from a previous bit position. Hence, it is only helpful for adding two single bits without chaining [1].

D. Full Adder

The full adder (FA) builds on the half adder by including C_{in} from the previous bit position. This adder combines three binary inputs into two output bits. Two out of three input bits are the operand bits from the current bit position and one bit as C_{in} coming from the previous bit position. Since this adder also takes the bit from the previous bit position, it forms an essential building block for multi-bit addition [1]. Several full adders can be cascaded to build circuits for adding multi-bit binary numbers.

TABLE III
TRUTH TABLE FOR FULL ADDER [1]

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Table III shows the truth table for a full adder with A, B, and C_{in} as input bits and S and C_{out} as the outputs. The boolean equations for the S and C_{out} are as follows [1]:

$$S = A \oplus B \oplus C_{\text{in}} = A'B'C_{\text{in}} + A'BC'_{\text{in}} + AB'C'_{\text{in}} + ABC_{\text{in}} \quad (3)$$

$$C_{\text{out}} = AC_{\text{in}} + BC_{\text{in}} + AB \quad (4)$$

The gate circuit for the C_{out} and S are given in Figure 5. Based

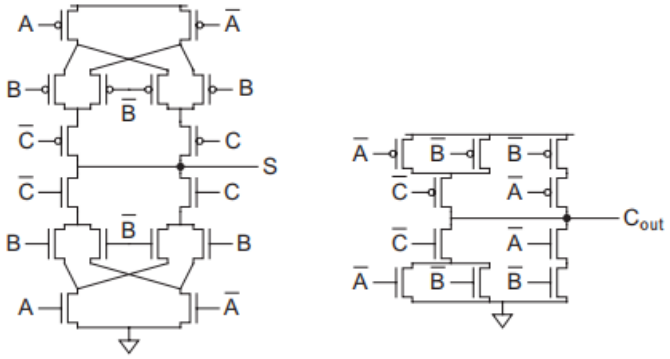


Fig. 5. Sum and C_{out} implementation for FA [2]

on the expressions for S and C_{out}, it is observed that S is a more complex function than C_{out}; hence it will have a bigger circuit because of long resistive paths and higher capacitive self-loading [1].

E. Ripple Carry Adder

The Ripple Carry Adder is one of the most basic and straightforward adder design for binary addition. A ripple carry adder is designed with one FA at the LSB and multiple FAs in subsequent bit positions. If LSB is assumed to have C_{in} value as 0, then HA can be seen as FA. In general, an N-bit ripple carry adder can be considered to have an N number of FAs and produce an output that has N+1 bits [1]. Figure 6 shows

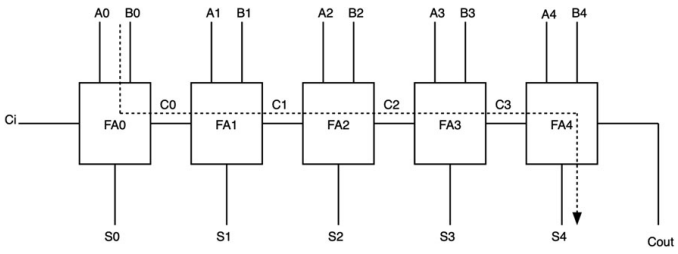


Fig. 6. A 5-bit ripple carry adder [1]

a 5-bit ripple carry adder with 5 FAs cascaded in a row. Each FA represents a bit position with LSB starting at FA0. Each adder adds the 2 bits from the operand at the given bit position along with the C_{out} from the previous bit position. The FA at the MSB (Most Significant Bit) produces S, and C_{out}, which are included in the final result, the same as in the conventional long addition method.

One of the most important metrics for designing an efficient adder is the delay. The circuit's datapath is comprised of these arithmetic circuits. For efficient performance, the circuit's datapath should be as fast as possible, with low power consumption and a smaller area [1].

In ripple carry adders, the delay behavior is observed in the sequential propagation of carry signals through the chain of full adders. Since in ripple carry, all the sums and carries are dependent on the previous carry values, creating a ripple effect, the critical path delay is calculated as the total time taken for the carries and the sum at the last bit [1].

The analysis of the delay in each FA is given below:

1) For the first FA, the C_{out} (C0) is ready after a delay of T_{carry}, while the sum (S0) is ready after T_{sum}. From the Subsection II-D, it can be observed that the $T_{\text{sum}} > T_{\text{carry}}$ since the expression of the S is much more complex.
2) The second FA begins computation only after receiving C_{out} (C0) from the first FA, introducing a cumulative delay. Its C_{out} (C1) is available at $2T_{\text{carry}}$, and its sum is available at T_{carry} + T_{sum}.
3) For the Nth full adder, the C_{out}. is available at NT_{carry}, and the sum is available at (N-1)T_{carry} + T_{sum}.

As the number of bits increases, the carry delay dominates the overall delay, while the sum delay only affects the final full adder. This behavior is evident from the linear dependence of delay on the number of bits, resulting in an O(N) time complexity [1].

Figure 7 shows the delay graph versus the bit count in the ripple carry adder. The delay of the sum (T_{sum}) affects only the y-intercept of the bit-versus-delay graph, while the carry delay (T_{carry}) defines the slope. For larger adders, T_{carry} becomes increasingly significant. Therefore, optimizing the sum delay for the last FA or reducing the carry delay can substantially improve performance. [1]

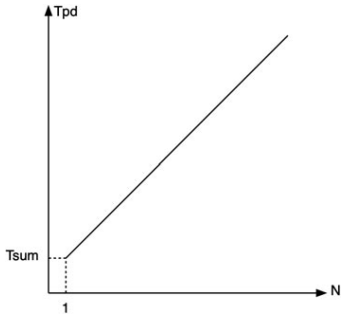


Fig. 7. Path delay vs Number of Bits [1]

III. Main Concepts

The linear dependence of delay on bit length in ripple carry adders necessitates optimization strategies for faster addition, especially in large adders. Techniques such as alternative adder architectures (e.g., carry-lookahead adders) can reduce the delay to a sublinear dependence, enabling higher-speed arithmetic operations in datapath circuits.

A. Generate Propagate Logic

The generate-propagate (G-P logic) approach is built upon the ripple carry adder and is designed to predict C_{out} value from C_{in}, based on the fact that in some cases C_{out} is either independent or tightly coupled with C_{in}. These cases are D (Delete), G (Generate), and P (Propagate) and are defined as:

- **Generate**: C_{out} is '1' regardless of C_{in}.
- **Delete**: C_{out} is '0' regardless of C_{in}.
- **Propagate**: C_{out} has the same value as C_{in}.

TABLE IV
Truth table for G-P logic [1]

A	B	C_{in}	C_{out}	S	G	D	P
0	0	0	0	0	0	1	0
0	0	1	0	1	0	1	0
0	1	0	0	1	0	0	1
0	1	1	1	0	0	0	1
1	0	0	0	1	0	0	1
1	0	1	1	0	0	0	1
1	1	0	1	0	1	0	0
1	1	1	1	1	1	0	0

Table IV is the truth table for Generate-Propagate logic using ripple carry adder. It is evident from the Table IV that G, P, and D variables are mutually exclusive, so only one variable has true value at a time [1]. The C_{out} value in terms of G, D, and P can be given as:

$$C_{\text{out}} = G + PC_{\text{in}} \tag{5}$$

The equations for G, D, and P in terms of input bits A and B are given as:

$$G = AB \tag{6}$$

$$D = A'B' \tag{7}$$

$$P = A \oplus B \tag{8}$$

It is evident from the Eq. 6 7 8 that G, D, and P are independent of C_{in} and only dependent on A and B. C_{out} is 0 when neither G nor P is true. Also, when P and G are false, D should be true, making D redundant in the calculation of C_{out} [1].

This mutual exclusivity of the G, P and D values and dependence only on A and B helps to calculate their values in parallel at operation initialization by adding a constant delay instead of the word length-dependent delay of the adder. Hence, the critical path delay is calculated as follows:

$$t_{\text{pd}} = t_{\text{gp}} + (N-1)t_{\text{carry}} + t_{\text{sum}} \tag{9}$$

Eq. 9 shows that the adder delay is linearly dependent on the N-word length. In Eq. 9, t_{pd} is the propagation delay, t_{gp} is generate-propagate delay, t_{carry} is carry delay and t_{sum} is the sum delay. For each adder, the C_{out} and S can be calculated from G-P logic as per Eq. 5 and 10, respectively.

$$S = A \oplus B \oplus C_{\text{in}} = P \oplus C_{\text{in}} \tag{10}$$

Figure 8 illustrates the block level implementation of generate-propagate logic using a 5-bit ripple carry adder. It can be observed that at each FA the G and P values are calculated at the start of the operation since input bits A_0 and B_0 are available at the start [1].

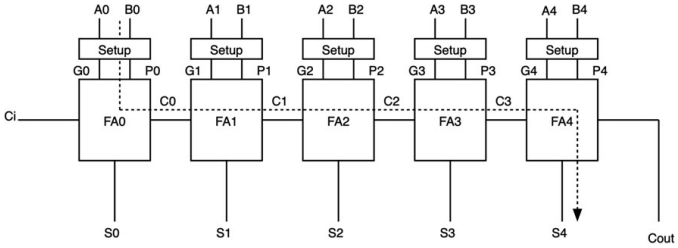


Fig. 8. G-P logic implementation using ripple carry adder [1]

The C_{out} equation is implemented using a Manchester carry chain. Figure 9 shows a Manchester carry chain for two cascading FAs. Each FA in the adder chain is a dynamic CMOS gate with an inverter, which gives C_{out}, which is further used as C_{in} input for the next FA [1].

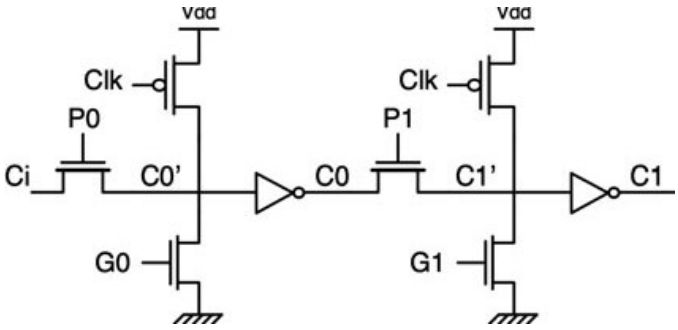


Fig. 9. Manchester Carry Chain [1]

Extraction of G and P from inputs A and B causes a minor increase in delay but a significant increase in the

circuit area, especially in propagate cases requiring two input XOR. The increased area and the Manchester gate results in power consumption similar to the normal FA carry gate [1].

Chaining multiple adders introduces an RC delay in propagating pass transistors. This will result in the delay increasing quadratically with respect to the increase in number of bits. The solution to the RC delay and to have linear growth is addition of static CMOS after each Manchester gate. This makes the design complex but is required for efficient ripple carry performance [1].
It is clearly evident that using a ripple carry adder with G-P logic and a Manchester chain does not improve the area, and the delay is comparable to traditional FA. However, this approach forms a foundation for designing faster adders, as it facilitates the observation and exploitation of carry chain behavior patterns in large blocks [1].

B. Carry Bypass Adder

The Carry-Bypass adder uses the generate-propagate logic to improve the design of binary adders. Similar to generate-propagate logic, the FAs are designed using Manchester carry chain architecture. This adder design is focused on bypassing C_{in} in a very specific case where every bit position is propagating. This adder is named so because C_{in} may bypass the entire adder to give carry-out [1].

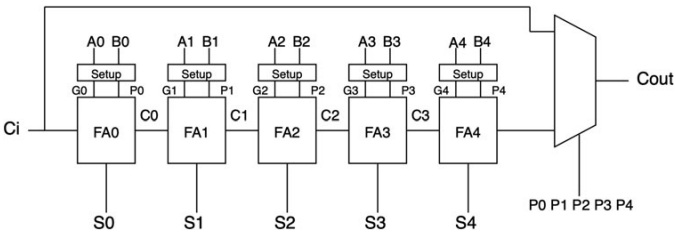


Fig. 10. Modified 5-bit ripple carry adder [1]

Figure 10 show a 5-bit ripple carry adder with G-P logic modified to bypass C_{in} to act as carry bypass adder. The final C_{out} of the adders is multiplexed in the following two cases:

- The whole adder is propagating, if propagates at all the bit positions are true, resulting in the same C_{out} as C_{in}.
- Else, the n-bit ripple carry output is C_{out}.

In a N-bit adder, the last output coming out is sum bit instead of C_{out} as the last sum bit takes more time than C_{out}. This shows that the final output is not ready until the sum is calculated [1].

All N-bit positions propagating in the adder is a special case. The improved delay with all bits propagating is only 1 out of 2^N time in an N-bit adder, while at other times, the C_{out} delay would be NT_{carry}. The best-case delay of an N-bit ripple carry adder is improved by bypass adder, but the worst case is still (N-1)T_{carry} + T_{sum}, resulting in a longer critical path.

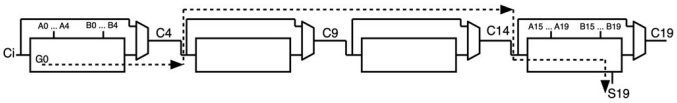


Fig. 11. Critical path for 20-bit carry-bypass adder [1]

Figure 11 shows the critical path delay for a 20-bit ripple carry adder. The delay is 19T_{carry} + T_{sum}, starting at bit 0 input position and ending at bit position 19 [1].

If the 20-bit adder is divided into four 5-bit blocks, block-wise bypass can significantly improve critical path delay. The worst-case scenario for carry propagation in a 20-bit adder cannot occur propagating bit positions, as this contradicts the established critical path.
The critical path analysis indicates that the carry generation starts at the first bit and is passed through the initial multiplexing to bits 1-4. In subsequent blocks, positions 5-14 also experience carry propagation, and it is noted that the last block focuses on S_{19} rather than C_{out}.
In intermediate blocks, if any bit is not propagating, it must either generate or delete a carry, making its C_{out} independent of the C_{in}, thus leading to a shorter calculation path starting from that bit. Therefore, all intermediate blocks, including the last, must be considered propagating irrespective of specific sum bits [1]. Hence, the maximum delay for a critical path for 20-bit adder can be calculated as follows:

$$t_{pd} = 5t_{carry} + 3t_{mux} + 4t_{carry} + t_{sum}$$

$$t_{pd} = Kt_{\text{carry}} + \left(\frac{N}{K} - 1\right) t_{\text{mux}} + (K-1)t_{\text{carry}} + t_{\text{sum}} \quad (11)$$

Eq. 11 gives a general equation of critical path delay assuming a N-bit adder having P blocks. The number of bits in every block can be calculated as $K = \frac{N}{P}$. In Eq. 11, t_{pd} is the propagation delay, t_{mux} is delay at multiplexer, t_{carry} is carry delay and t_{sum} is the sum delay.

Eq. 11 shows a linear delay dependence on the number of bits, expressed about the number of bits per block K, resulting in a reduced slope that indicates improved delay performance, summarized as $O(N/K)$ [1]. Figure 12 shows the graph of carry bypass delay vs the number of bits, N.

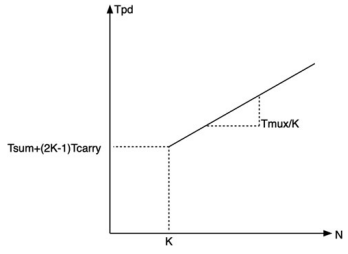


Fig. 12. Carry-Bypass delay with N [1]

It can be observed that the choice of block size K in this adder design greatly influences performance. While increasing

K can improve the slope of delay, making the adder faster for large N, it can also lead to sub-optimal configurations if K is equal to N, reverting the design back to a ripple carry adder [1].
Figure 12 shows that the intercept term is influenced by K. As K increases, the intercept, given as $T_{sum} + (2K-1)T_{carry}$, also increases, while the slope decreases. For small N, a lower K helps mitigate the initial penalties at the y-intercept. For larger N, a higher K is preferred to reduce accumulated delays due to lower slope. At the extremes, if $K = 1$, it results in a standard ripple-carry chain with each block consisting of one bit, while $K = N$ leads to a single block behaving as a ripple-carry adder [1].

The optimal K can be determined by solving the inflection point of the delay expression, indicating that both increasing N and the ratio of multiplexer delay to carry delay encourage a higher K to minimize overall delay. The optimal value for K is given as:

$$K_{\text{opt}} = \sqrt{\frac{N \cdot t_{\text{mux}}}{2t_{\text{carry}}}} \tag{12}$$

From Eq. 12, it is clear that K_{opt} is high with increase in N and a higher $t_{\text{mux}/t_{\text{carry}}}$ ratio increases K. This indicates that multiplexers have greater delay relative to the carry chain. Hence, this adder design should use fewer multiplexers, fewer blocks, and more bits per block while satisfying the condition $K \leq N$ for optimal design [1].

C. Carry-Save Adder

The carry-save adder is designed to improve the linear dependence of delay on the number of bits, N, by reducing the slope in comparison to the ripple carry adder. This area focuses on the improvement of speed, with an increase in area as a drawback. It uses two carry chains in each block, allowing for more efficient calculations as both C_{out} scenarios (given C_{in} as "0" or "1") are evaluated simultaneously across a more significant number of N-bit adders. This can be designed using either ripple carry adder or generate-propagate logic in Manchester carry chains. Carry-save architecture helps to start calculations simultaneously, providing the availability of conditional carry outputs at the multiplexers. The carry-out calculated for the first block influences the next block's carry-outs sequentially [1].

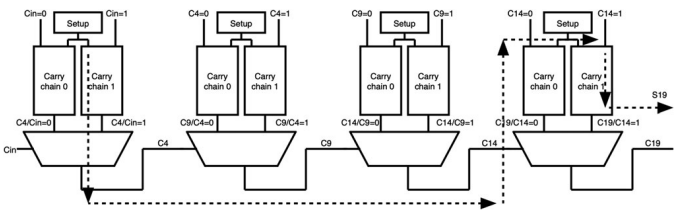


Fig. 13. Carry Save adder design [1]

Figure 13 shows the critical path of the 20-bit carry-save adder with 5-bit blocks. For a 20-bit adder, there are eight 5-bit adders used and the calculation starts at the beginning of the operation. The C_4/C_{in} at bit position 4 is C_{out} = 0 if the C_{in} to the block is 0 and 1 if the C_{in} is 1 [1].

Once the C_{out} for the first block is calculated, it chooses either of two C_{out} of the second block, and then the C_{out} from the second block chooses one C_{out} from the third block and so on. This ensures that the C_{out} and the selector propagates through the whole chain. The inputs to the multiplexer are calculated parallely at the start of the operation and are present once the carry arrives at the multiplexer [1]. A generalized equation for the critical path in case of carry-save adder is given as:

$$t_{pd} = Kt_{\text{carry}} + \left(\frac{N}{K} - 2\right) t_{\text{mux}} + (K-1)t_{\text{carry}} + t_{\text{sum}} \tag{13}$$

Though this adder design reduces its delay slope, it does not achieve sublinear delay. This issue is encountered because the carry outputs from intermediate blocks become available sooner than needed due to equal block sizes, leading to a wait during multiplexing. To resolve this issue, blocks of variable sizes can be used, which can allow bigger subsequent blocks to synchronize their outputs with the selector lines more efficiently. The design suggests that if the delays of multiplexers and carries are equal, block sizes can be incrementally adjusted to optimize carry input readiness at multiplexers [1]. Hence, the critical path delay equation can be rewritten as:

$$t_{pd} = Kt_{\text{carry}} + \left(\sqrt{\frac{2N}{M}} - 2\right) t_{\text{mux}} + (K-1)t_{\text{carry}} + t_{\text{sum}} \tag{14}$$

$$\text{where} \quad M = \left\lfloor \frac{t_{\text{mux}}}{t_{\text{carry}}} \right\rfloor$$

Eq. 14 is written assuming P stages and the first stage has K bits. The subsequent stages have bit number increasing by 1. This approach allows for a general formula representing the total number of bits and the critical path delay, potentially achieving a sublinear dependence on N, especially for large configurations with M reflecting the relationship between multiplexer and carry delays [1].

D. Lookahead Addition

The adder designs in the previous sub-sections have a common focus area to improve the design which is to reduce the dependence of the delay on the number of bits to be processed. In order to design fast adders, an ideal design would be to make the delay independent of bit number N. The ripple carry adder and the carry-save adder have linear and square root dependence on N, respectively [1].
The lookahead adder concept on the surface aims to make the delay independent of the number of bits. For the lookahead adder of bit count N, the carry bit can be calculated as follows:

$$C_{\text{n}} = G_{\text{n}} + P_{\text{n}}C_{\text{n-1}}, \quad \text{where } n = 0 \text{ to } N-1 \tag{15}$$

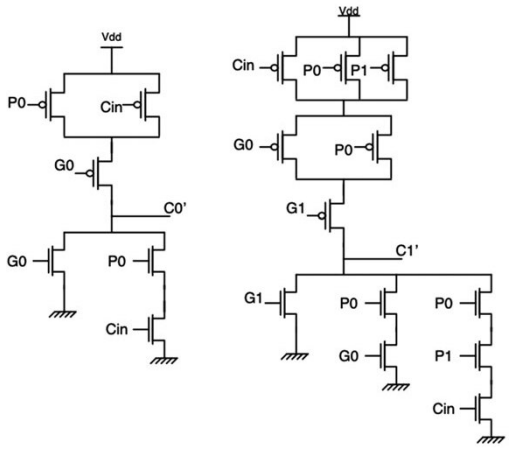


Fig. 14. Lookahead Adder for bit position 1 and 2 [1]

$$C_{n-1} = C_{\text{in}} \quad \text{for } n = 0 \tag{16}$$

For a bit position n, the carry can be expanded to be independent of the previous carry as follows:

$$C_n = G_n + \sum_{i=0}^{n-1} (P_n P_{n-1} \dots P_{i+1} G_i) + P_n P_{n-1} \dots P_0 C_{\text{in}} \tag{17}$$

In Eq. 17, C_{n}, P_{n} and G_{n} are nth carry, propagate and generate values respectively. From the Eq. 17, a false sense of independence from the bit count N is perceived since the carry calculation can be started as the operation starts. In reality, the static CMOS circuits of the latter carries cause longer delays due to the increasing complexity of carry expressions. Specifically, the NMOS pull-down network length and number of connected drains at the circuit output increase with N, increasing resistance and capacitance with O(N) each. This results in the time constant at the output node increasing as O(N^2) [1].

The last term in the Eq. 17 dominates the delay of the nth carry position, which dominates the circuit's behavior. Since a Manchester chain allows extracting all the carry position values simultaneously, propagate logic can be implemented using this. However, a Manchester carry chain adds quadratic dependency on delay unless static CMOS buffers are added. Without buffers, the lookahead adder delay resembles that of a ripple carry adder [1].

Figure 15 show the lookahead logic to calculate carry positions. It is observed that the carry expression can also be seen as the sum of products. These are designed using the first layer of NAND gates along with a NAND gate. The product with the longest chain will always be the one for MSB, and the initial layer NAND gate to calculate this term will have the most extended delay leading to the critical path [1].

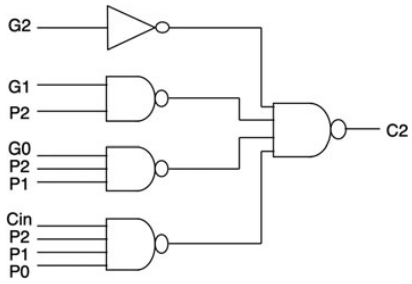


Fig. 15. Lookahead logic [1]

The time constant in case of lookahead addition will grow linearly since the capacitance and resistance do not increase simultaneously. This is because in the PDN (Pull Down Network) of the NAND gate, with an increase in the number of inputs, path resistance increases if the transistor size is kept the same, or capacitance at the output node increases if the transistor size is increased [1].

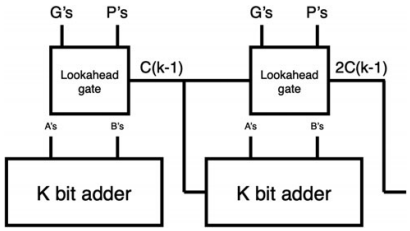


Fig. 16. Lookahead Adder: Implementation for large adder with blocks [1]

Figure 16 shows an example of a 2K-bit adder which can be split into two K-bit blocks. The first block uses lookahead logic to compute the K-1 carry quickly, while internal carries use ripple logic. The C_{out} of the first block becomes the C_{out} for the second block [1].

The lookahead adders depend on N linearly or quadratically, but their simple logic enables them to calculate the latter carry positions at the operation start. This is advantageous for small bit sizes due to the low initial delay (y-intercept). This property is helpful in large adders by dividing them into smaller bit groups, and the most important carry value in a block is calculated using the lookahead principle [1].

The hierarchical application of lookahead arithmetic allows for designing fast adders. Even if their dependence is quadratic on N, lookahead adders remain crucial for efficient adder designs in practical implementations [1].

IV. Current Status

The current scenario of binary arithmetic for digital circuit design is focused on performance improvement in terms of factors like efficient power consumption, small circuit area size, high speed, and the ability to scale the design for longer word lengths.

Ripple carry adders provide a very simple circuit design for smaller word lengths with small area utilization. The ripple carry adder's circuit delay is influenced greatly by the bit length of the operands. Ripple carry adders are still one of the most cost-effective adder designs in binary arithmetic [3]. Another type of binary adder are carry lookahead adders and their variants, which rely on processing the carries in a parallel or hierarchical way to reduce critical path delays. These adders are mostly based on techniques such as generate-propagate logic and Manchester carry chain design to calculate the carry-out values at the start of the addition operation. These designs are more suitable for long word lengths since they perform the addition operations on different bit positions by dividing the original word into several blocks. The drawback that comes with these designs is an increase in the circuit area, which results in higher power consumption. Thus, the focus of research in this area is reducing the area and improving the power dissipation [3].

V. Summary

To summarize, this report elaborates on the advanced architectures for binary addition. The ripple carry adder provides a straightforward, area-efficient design for binary addition using full adders. The main drawback of the ripple carry adder is the long propagation delay due to its linear dependence on the number of bits. Advanced design techniques were discussed, which help to decrease the dependence of the number of bits on the delay using hierarchal and parallel techniques. The generate-propagate logic builds upon the ripple carry adder and forms the basis of the carry lookahead design of adders by generating carries in parallel. The Carry Bypass adders technique helps to decrease the critical path delay by dividing the total bit length processes into blocks. With this design, several intermediate blocks are bypassed to lower the delay, which would otherwise be very large due to the sequential nature of the ripple carry adder. Carry Bypass design provides faster adder results, especially when adding operands of long word lengths. Carry save adder provides an efficient solution for multi-operand addition as, unlike conventional adders, it avoids immediate carry propagation. This helps to reduce the computation time and improve performance. The Lookahead adder design computes carries parallel in order to minimize the delay. However, the circuit becomes more complex, which leads to higher power consumption.

References

[1] Abbas, Karim. "Arithmetic." In Handbook of Digital CMOS Technology, Circuits, and Systems, edited by Karim Abbas, 439–51. Cham: Springer International Publishing, 2020. https://doi.org/10.1007/978-3-030-37195-1_11.

[2] Weste, Neil H. E., and David Money Harris. CMOS VLSI Design: A Circuits and Systems Perspective. 4th ed. Boston: Addison Wesley, 24-88, 2011.

[3] "ChatGPT." Accessed January 25, 2025. https://chatgpt.com.